Brain network: the rules of the notes and the cost of the time steps

Date: October 30, 2026 · **Sources**: the notes of `.info` (the rules and the cost question), `src/brain/network/*.cpp`, a benchmark written for this report (brain network vs layered network of the same number of weights, `perf` on the execution) · No code was changed.

> **How to read this document**
>
> 1. Which rules of the notes the library respects today (part 1).
> 2. Why the brain network is slow next to a layered one, measured (part 2).
> 3. The ways to reduce that cost, with their gain, their price and what they break (part 3).
>
> **Limits**: the machine was loaded (load average 9 to 10), each number is one run of a loop of 200 to 3000 executions, so the differences under 1.5x are noise. The gains marked *estimate* are computed from the measured cost of one link and one activation, they are not measured. Everything is on the host (RAM), one sample at a time.

Contents

1. Summary
2. The rules of the notes
3. The cost of the time steps
4. Solutions
5. Recommended order
6. Reproduce

1. Summary

Point	Conclusion
Rules	8 ideas: **4 done** (at least one link per neuron, inputs only send, outputs only receive, no break of the last path: this one only when asked), **2 partly** (the path of every input at the init, the bias as a threshold), **2 not done** (start at 0 links, move the last weak link elsewhere)
The cost is not exponential	one execution costs `steps x links` (about `n^1.5 log n`), a layered network of the same weights costs `links` : the brain network is `steps` times more work by design (21 to 34 steps measured)
Measured gap	26x (339 weights), 92x (1700), 180x (12 000) for the execution; 44x, 122x, 389x for one training sample. The gap grows faster than the number of steps: 3 to 5x more per link than a layered network (gather instead of a contiguous product) and an activation per neuron **per step**
Half of the execution time is the activation	`expf` + `sigmoid` = 49 % of the execution (profile), the sum of the links the other 51 %
Reaching the layered cost	only possible without recurrence: a single pass in the order of the positions (`leftToRight`), *estimate* 1.5 to 4x a layered network. With recurrence (memory, sequences) the floor is `steps` layered passes, like any recurrent network
Best first steps	steps = the hops needed (4 instead of 27: **x6.6 measured**), a fast sigmoid (*estimate* x1.3), early stop when the states stop changing, then the single pass mode

2. The rules of the notes

#	Idea of the notes	Status	What the library does	Default
1	The bias is the threshold that launches the signal, and only this upgrades the network	**Partly**	The bias is added to the sum before the activation: a soft threshold (sigmoid) that moves where the neuron answers. A hard threshold also exists (`threshold(value)` : a signal under it is cancelled). The weights are trained too: the bias is not the only thing that changes	bias trained, hard threshold `BRAIN_SIGNAL_THRESHOLD`

#	Idea of the notes	Status	What the library does	Default
2	Everything starts at 0 links, how do new links appear?	**Not done**	The init gives each neuron **a random number of links** between `minLinks()` and the cap (before: as many as the cap). A neuron never starts with 0. New links appear when a weight reaches `weightMax` (the nearest free neuron is linked)	random count, minimum 1
3	Under the threshold with one link: change that link and link it to another neuron with a low weight (5 % of the limits)	**Partly**	A link at `weightMin` is removed, **except the last link of a neuron, which is kept** (it is not moved). A new link starts at 5 % of the limits when `lowNewLinks(true)` (else between half and all of the mean of its two nearest links). The *move* itself (cut the last weak link and link the neuron elsewhere) does not exist	`lowNewLinks` off in the class, on in the lab
4	Never break the last link between an input and an output	**Done, on request**	`keepPaths(true)` : a removal that takes the last path of an input to an output away is put back (tests)	off in the class, on in the lab
5	Every input has a path to the output	**Partly**	`pathInit(hops)` builds a path of at most `hops` links from every input to every output at the init (a crash of it was fixed this week). Only at the init: during the training `keepPaths` stops it from being cut, nothing builds a new one	off in the class (0), on in the lab
6	At least one link on each neuron	**Done**	A neuron never loses its last link, nor the last link of its kind (an input keeps an outgoing link, the others an incoming one) unless `allowOrphans()` **and** `minLinks(0)` . `minLinks(n)` generalizes it	on (`minLinks` 1)
7	Inputs only have outgoing links	**Done**	Enforced at the init, when a link is added, and a file where an input receives a link is refused (`InvalidNetworkFile`); tests	always
8	Outputs only have incoming links	**Done**	Same as 7 for the outputs	always

What is new compared to the notes: the minimum of links (`minLinks`), the random number of links at the init, the inverted mode (cut at `weightMax` , grow at `weightMin`) and the paths of a left-to-right network.

> **Key takeaway** The two missing behaviors are small: *start at 0 links* needs a `randomLinks` minimum of 0 and an init of the paths only, and *move the last weak link* is one branch in `relink_` (instead of keeping it, remove it and link the neuron to its nearest free neuron at 5 % of the limits). Both change the training, so they should be measured on the scenarios of the lab before becoming defaults.

3. The cost of the time steps

3.1 Not exponential, but `steps` times more

One execution of a brain network does, at **each of the `steps` steps**, a sum over the links of **every** neuron and an activation of every neuron:

	Layered network	Brain network
Multiply-adds	`W` (the weights)	`steps x W`
Activations	one per neuron (`N`)	`steps x N`
Default size rule	the user's layers	links `3 sqrt(n)` , steps `4 log2(n)`
Training	one backward pass	one backward pass **per step** (BPTT), the states of every step kept

With the default caps the work grows like `n^1.5 log n` : a polynomial, not an exponential. It *feels* exponential because both factors grow together: more neurons means more links per neuron **and** more steps.

3.2 Measured

A brain network and a layered network `{h, h}` with the same number of weights, host, 8 inputs and 2 outputs (one run each):

Hidden neurons	Weights	Steps	Brain exec	Layered exec	Gap	Brain train	Layered train	Gap
30	339	21	30.7 us	1.2 us	**26x**	107.9 us	2.4 us	**44x**

Hidden neurons	Weights	Steps	Brain exec	Layered exec	Gap	Brain train	Layered train	Gap
100	1 696	27	169.2 us	1.8 us	**92x**	475.2 us	3.9 us	**122x**
400	12 264	34	1 972.5 us	11.0 us	**180x**	7 252.9 us	18.7 us	**389x**

With only 4 steps (`stepsCap` 4) the same networks give:

Hidden neurons	Brain exec, 4 steps	Gain	Gap to layered	Brain exec, 4 steps and `leftToRight`	Brain train, 4 steps
30	8.1 us	x3.8	6.9x	5.8 us	33.1 us
100	25.7 us	**x6.6**	13.5x	33.1 us	137.4 us
400	208.7 us	**x9.4**	26x	263.6 us	2 009.9 us

> **Key takeaway**
> Cutting the steps from the default (21 to 34) to 4 is the cheapest gain: 4x to 9x on the execution. It is also where the lab scenarios already are (8 steps solved all four). The remaining gap (7x to 26x) is the cost per link and per activation, not the steps.

3.3 Why the gap is bigger than the number of steps

At 1 696 weights the gap is 92x for 27 steps: **3.4x more per operation** than the layered network. The profile of the execution (100 hidden neurons):

Share	Function	Meaning
26.6 %	`BrainNetwork::compute_`	the sums over the links: for each link a read of its id, its weight and the state of the sender at random places (gather)
14.9 %	`expf`	the exponential of the sigmoid
10.6 %	`tools::sigmoid`	the division and the call around it
9.5 %	`RawBrainNetwork::apply()`	the forward pass of the training
7.3 %	`learn_`	the backward pass
6.0 %	`sync_`	the reduced copy rebuilt after **every** update
5.1 %	`memmove`	copies of the states and of the memories
3.8 %	`update_`	the weights update

For the execution alone, the activation (`expf` + `sigmoid`) is **49 %** and the links 51 %: about 31 ns per activation and 1.9 ns per link, against about 1 ns per link and 10 ns per activation in the layered network (contiguous weights, vectorized). The brain network evaluates `steps x N` activations (2 700 here) where the layered one evaluates 72.

3.4 What equal cost means

- A brain network **with recurrence** does `steps` passes through its links. The fair comparison is a recurrent network unrolled `steps` times, not a layered network: the cost is the same by construction. The floor of the gap is `steps` , minus what skipping and layout can win.
- A brain network **without recurrence** (`leftToRight` : a link only goes from the left to the right, so no loop) can be computed **in one pass** in the order of the positions: the same work as a layered network. This is the only way to be *equivalent*, and it removes the memory (the sequences of the MEMORY scenario need the recurrence).

4. Solutions

Gain is on the execution at 1 700 weights unless said otherwise; **measured** = in section 3, *estimate* = computed from the cost per link (1.9 ns) and per activation (31 ns).

#	Solution	Gain	Training	What it changes	Effort	Risk
1	**Steps = the hops needed.** Set the default steps from the longest shortest path input -> output (the lab already filters on it: 3 hops), plus a margin, instead of `4 log2(n)`	**x6.6 measured** (27 -> 4 steps)	same factor	long chains and memory need more steps	S	low: a default
2	**Single pass mode** for a `leftToRight` network: the neurons sorted by position once, states computed in order, in place, no steps	*estimate* 1.5 to 4x a layered network (brain 1 696 weights: about 1 700 links x 1.9 ns + 110 activations x 31 ns = 6.6 us; with 5 and 6 below 3 us)	one backward pass, **no BPTT**, no states per step: **x20 to x30** less	no recurrence (no memory)	M	medium: a second execution path, same results as one step per level
3	**Early stop** when no state changes by more than a tolerance between two steps (a feed-forward part settles after its depth)	*estimate* x2 to x5 on a network that settles	same, the backward pass starts at the last step done	results equal to the tolerance, not bit for bit	S	low
4	**Fast activation**: vectorized `expf` , or a rational / table sigmoid (about 3 to 5 ns)	*estimate* x1.3 to x1.8 (the activations are 49 %)	same on the forward pass	last digits of the outputs	S	low: tests need a tolerance
5	**Packed links** (structure of arrays): ids on 32 bits and weights in two contiguous arrays (8 bytes per link instead of 16, no `_inSlot` indirection), the states as one array	*estimate* x1.5 to x2.5 on the sums	same	none visible	M	low
6	**SIMD / vectorized sums**: gather instructions or sorted ids per neuron (nearest first = close in memory)	*estimate* x1.5 to x3 with 5	same on the backward pass	none visible	M	low
7	**A batch of samples**: `apply(Dataset)` computes B samples at once (states as a matrix `N x B`), the links are read once for B samples and the inner loop over B is contiguous	*estimate* x4 to x8 per sample on CPU; on GPU it is what fills the device (today one sample = one tiny kernel per step)	the mini-batch already gives B gradients: they can be one pass	new API	M	low
8	**The update without the rebuild**: `sync_` (6 %) and the copies (5 %) after every update; rebuild the reduced copy only when `reduce()` / the execution of a const network needs it, update the changed slots only	*estimate* about 10 % of the training	**x1.1 train**	none	S	low
9	**Skip the silent links**: with `threshold` and ReLU, many states are exactly 0 (not with a sigmoid, which is never 0): skip a link whose sender is 0	depends on the sparsity; none with a sigmoid	same	none	S	low, useless on a sigmoid
10	**GPU: one kernel for all the steps** (persistent kernel with a grid-wide barrier, or CUDA graphs) and the batch of 7: today the GPU needs one launch per step and loses to the host on small networks (see the benchmark of the 25th)	x3 to x10 on small networks, none on large ones	same	the warp kernels of this week stay	L	medium
11	**Truncated BPTT**: go back only `k` steps from the output instead of all of them	train x(steps / k)	less exact gradients	gradient	S	medium: measure on the scenarios
12	**Threads** over the neurons of one step	*estimate* x2 to x4 above 1 000 neurons, loses below (a barrier per step)	same	none	M	low

> **Key takeaway** Only solution 2 makes the brain network *equivalent* to a layered one, and only without recurrence. Everything else cuts the gap without closing it: 1 + 3 + 4 + 5 + 6 should bring the recurrent network from 92x to about 5x to 10x of a layered network at 1 700 weights (*estimate*, to be measured).

5. Recommended order

Step	Do	Why first
1	Solution 1 (steps from the hops) and 8 (no rebuild per update)	a default and a removal: almost no code, 6.6x measured on the execution
2	Solution 4 (fast activation) and 5 (packed links)	the two halves of the execution time, no change of behavior
3	Solution 7 (batch of samples)	the one that also makes the GPU useful; the mini-batch of the training already wants it
4	Solution 3 (early stop)	cheap, depends on the networks that settle: measure first
5	Solution 2 (single pass `leftToRight`)	the only equivalent of a layered network: decide it with the scenarios (DIGITS, WORDS, TICTACTOE do not need the recurrence, MEMORY does)
6	Solution 10 (GPU)	after the batch, otherwise the launches keep dominating

Each step is measured with the same comparison (brain vs layered of the same weights, 30 / 100 / 400 hidden neurons), and the checksums of the unit tests must stay (a tolerance for 4 and 6).

6. Reproduce

The benchmark of this report is a small program linked on `libbrain.a` (`brain::RawBrainNetwork` against `brain::LayerNetwork` of the same number of weights): for each size and each variant (default steps, 4 steps, 4 steps `leftToRight`, fewer links) it times 200 executions and 50 training samples, and prints the work done (`steps x weights`). The profile is `perf record -F 2000` on the same program (100 hidden neurons, 3 000 executions). It is not in the repository; say if it should be added to `tests/` or a `bench/` folder.